

```
<artifactId>ums-add-service</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>ums-add-service</name>
<properties>
  <java.version>1.8</java.version>
</properties>
```

From this snippet, you can find that we are on the way to building an artifact named *ums-add-service* under the group *com.glarimy*. It is also clear that we prefer version 2.7.0 of Spring Boot. Different versions of Spring Boot support different versions of Java, and this one supports the time-tested version 1.8.

Now let's explore the dependencies.

The *spring-boot-starter-web* adds an embedded Tomcat Web server along with HTTP-based MVC framework and REST capabilities.

The *spring-boot-starter-data-rest* adds the capabilities to generate the repositories (recollect the Repository pattern of domain-driven design). The repositories can be backed by various storage technologies.

The *spring-boot-starter-data-jpa* adds capabilities to connect to JDBC data sources using JPA API on top of Hibernate ORM implementation. With this, Spring Boot automatically chooses JPA to build the repositories. However, the JPA must be backed by an RDBMS. The *h2* artifact adds the H2 in-memory all-Java RDBMS. You may choose any other database like MySQL to support JPA. We want to keep it simple and hence chose this embedded database server, for now!

The *spring-boot-starter-validation* adds the validation framework on top of the Hibernate Validator libraries.

The *spring-boot-start-test* adds test capabilities. The *spring-boot-devtools* adds a few useful development tools. For example, one of the tools watches the source code, and automatically rebuilds and reruns the whole service every time you edit the code.

The following build section instruments the Maven plugin that is specifically developed for Spring Boot.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
```

It generates a fat JAR with compiled class files, libraries and, most importantly, the embedded Tomcat Web server.

That's all about *pom.xml*. Let us move on to the code.

The domain model

The domain model we developed in the previous part consists of the classes *Name*, *PhoneNumber*, *User*, and *UserRepository*. These fundamentally remain the same.

However, we are decorating them to inject a few capabilities of Spring Boot like validation and persistence. This reduces the amount of code that we otherwise have to write.

Recollect that the *User* is modelled as an *Entity* class on the lines of domain-driven design. The *AddService* is supposed to persist the *User* objects using *UserRepository* (into H2 RDBMS, in this case). Instead of writing a lot of code for that, we decorate the *User* with JPA annotations and generate the *UserRepository* with the help of Spring Boot.

Here is the annotated *User* class:

```
@Entity(name = "ums_users")
public class User {
    @EmbeddedId
    @Valid
    @NotNull
    private Name name;
    @Embedded
    @Valid
    @NotNull
    private PhoneNumber phone;
    private Date since;
    User() {}
    ...
}
```

With the *@Entity* annotation, we are mapping the objects of the *User* to the rows of a table named *UMS_USERS*. The properties of *Name* act as the columns of the primary key because of the *@EmbeddedId* annotation. And the properties of *PhoneNumber* are included in the *UMS_USERS* table because of *@Embedded* annotation. In other words, the properties of *User*, *Name*, and *PhoneNumber* are all part of a single table named *UMS_USERS*.

The *@Valid*, *@NotNull* annotations come from the validation framework. The *@NotNull* annotations make sure that the name and phone are not null. And *@Valid* annotation makes sure that *Name* and *Phone* objects are also validated.

It can be observed that we also added a default constructor to the *User* class with package visibility to satisfy the JPA requirements. However, it still sticks to the domain model. Remember that the domain model is technology-agnostic. It should remain the same with or without Spring Boot.

The following is the decorated *Name* class:

```
@Embeddable
public class Name implements Serializable {
    private static final long serialVersionUID =
-5928119910862624448L;
    @Column(name = "name")
    @Size(min = 8, max = 32)
```